SE 1105 - PROGRAMMING AND PROBLEM SOLVING I

Preprocessors, Random Number Generator, Mastermind Game

Asst. Prof. Dr. Kazım Erdoğdu

Faculty of Engineering
Department of Software Engineering
Yaşar University

Fall, 2024

Table of Contents

1. Preprocessors
   - #include
   - #define
   - Predefined Macros
2. Random Number Generator
3. Case Study - Mastermind Game

Table of Contents

1. Preprocessors
   - #include
   - #define
   - Predefined Macros
2. Random Number Generator
3. Case Study - Mastermind Game

Preprocessors

- Preprocessors are directives that are executed before the compilation process.
- They begin with # symbol.
- They are used for
  - including other files to a program (**#include**)
  - defining constants and macros (**#define**)
  - conditionally compiling a program (**#if, #else, #elif, #ifdef, #ifndef, #endif**)
  - prompting error messages (**#error**)
- We will focus only on the first two.

#include

- Hence the name, **#include** includes the contents of another file in the current program.
- e.g. #include <stdio.h>, #include "myHeader.h"
- In the above examples, the former searches for the header file in the system folder, while the latter searches for the header file in the current folder.

#include

arrayOps.h

```c
int sum(int A[], int size){
  int sum = 0;
  for(int i=0; i<size; i++){
    sum = sum + A[i];
  }
  return sum;
}
```

main.c

```c
#include <stdio.h>
#include "arrayOps.h"

void main(){
 int A[] = {-3, 7, 11, -9};
 printf("Sum:%d\n",sum(A,4));
}
```

Exercise: Let us create our header file that contains functions about finding the sum, average, min and max of integer arrays.

#define

- Hence the name, **#define** preprocessor is used for defining macros:
  - defining constants

```
#define PI 3.14

#define SIZE 100
```

#define

- defining functions

```
#define SQUARE(n) n*n

#define CIRCLE_AREA(r) PI*SQUARE(r)

#define MULT(a,b) a*b

#define LOOP(a, b)                          \
    for (int i = a; i <= b; i++) { \
        printf("%d\n", i);                  \
    }
```

Predefined Macros

- We can get the following information by the predefined macros in C:
  - __FILE__ : the name of the source C file.
  - __DATE__ : the date the current C file is compiled.
  - __TIME__ : The time the current C file is compiled.
  - __LINE__ : the current line in the code.
  - __STDC__ : the standard C support check. If the compiler supports the standard C it returns 1, otherwise it returns 0.

Predefined Macros

```
1 #include<stdio.h>
2
3 void main(){
4     printf("File: %s\n", __FILE__ );
5     printf("Date: %s\n", __DATE__ );
6     printf("Time: %s\n", __TIME__ );
7     printf("Line: %d\n", __LINE__ );
8     printf("STDC: %d\n", __STDC__ );
9 }
```

Table of Contents

1. Preprocessors
   - #include
   - #define
   - Predefined Macros
2. Random Number Generator
3. Case Study - Mastermind Game

Random Number Generator

- Pseudo-random number generation (PRNG) is a field of study in itself.
- It requires algebra, statistics, and cryptography.
- Generating a random number on computers is not a random process because compilers or programs use algorithms to imitate random number generation. (An example list of these algorithms can be found at https://en.wikipedia.org/wiki/List_of_random_number_generators#Pseudorandom_number_generators_(PRNGs))

Random Number Generator

- In this course, we will not deal with that part of PRNG.
- We will use a predefined random function called **rand( )** from **stdlib.h** library in C.
- rand() function does not take an argument and returns an integer number between 0 and RAND_MAX (i.e. 32767).

```
1    int r = rand();
```

Random Number Generator

- To generate a number between **0** and **n**:

```
int r = rand() % (n+1);
```

- To generate a number between **a** and **b**:

```
int r = a + rand() % (b-a+1);
```

Random Number Generator

- Here is a program that simulates rolling a die 100 times:

```
#include<stdio.h>
#include<stdlib.h>

void main(){
  for(int i=0; i<100; i++){
    print("%d, ", 1 + rand() % 6);
  }
}
```

Random Number Generator

- What happens if you run this program again and again? You do not get different results.
- The reason is that the starting value of the randomization algorithm never changes.
- Dynamically changing the initial value (called the **seed**) will change the results each time you run the program.
- We need to use these two functions:
  - **srand( )** from **stdlib.h**
  - **time( )** from **time.h**

Random Number Generator

```
1   #include<stdio.h>
2   #include<stdlib.h>
3   #include<time.h>
4
5   void main(){
6       srand(time(NULL));
7       for(int i=0; i<100; i++){
8           print("%d, ", 1 + rand() % 6);
9       }
10  }
```

Exercises

1. Write a C function that rolls a die 100 times and prints out the frequencies of each number being rolled.

2. Write a C function that generates a random number between a and b (passed to the function by value) and assists the user in guessing this randomly generated number by giving "UP" and "DOWN" directions.

Table of Contents

1. Preprocessors
   - #include
   - #define
   - Predefined Macros
2. Random Number Generator
3. Case Study - Mastermind Game

Case Study - Mastermind Game

Figure: https://en.wikipedia.org/wiki/Mastermind_(board_game)

```
Guess 1) 1234 --> +-
Guess 2) 5678 --> -
Guess 3) 9027 --> ++-
...
```

Figure: Break the Code

Case Study - Mastermind Game

Brief Explanations

- There is a hidden code with non-repeating digits.
- The user is to find that number by making guesses.
- After each guess, the program returns number of $+$ and $-$ signs to the user.
- Each $+$ means that one of the digits in the user's guess matches with one of the digits in the hidden code in the correct location.
- Each $-$ means that one of the digits in the user's guess matches with one of the digits in the hidden code but NOT in the correct location.
- The game is over when either the code is broken or the max number of guesses is wasted.

LET'S CODE IT IN C!!!